ScopeFlow — Backend Design (Node.js / JavaScript)

Version 0.1 · 13 Oct 2026 · Scope: Week 1 (13–19 Oct), full workflow with no AI agents

1. Purpose

This document defines how the ScopeFlow backend is structured so that business logic, persistence, payments and verification are **separate and loosely coupled**. Week 1 delivers the entire workflow (project → agreed spec → funding → submission → review → payment) using human review and CI results only. Auditing agents arrive later as new implementations of an existing interface, with no changes to the core.

Goals

- Domain rules live in plain JavaScript classes that know nothing about HTTP, SQL, PayPal or GitHub.
- Every external system sits behind an interface (a "port") with a real adapter and an in-memory fake.
- Modules communicate through use cases and events, never by reading each other's tables.
- The whole system can run and be tested offline.

Non-goals for week 1: agents, LLM calls, mutation testing, production payouts, microservices.

2. Technology choices

Concern	Choice	Reason
Runtime	Node.js current LTS, ES modules	Standard, long support window
Language	JavaScript with JSDoc type annotations and `// @ts-check` in editors	Stays JavaScript but catches wiring mistakes; easy migration to TypeScript later
HTTP framework	Fastify (Express is an acceptable alternative)	Schema validation built in, fast, clean plugin model
Database	PostgreSQL (SQLite acceptable for local dev)	Transactions, constraints, JSONB for evidence metadata

Concern	Choice	Reason
Data access	Knex (query builder) with `pg`, migrations via Knex	Explicit SQL control keeps ORM objects out of the domain; Prisma/Sequelize would blur the domain boundary
Validation	Zod (or Fastify JSON schema) at the API edge	Domain stays free of request-shape concerns
Auth	Email/password with bcrypt/argon2 + signed session or JWT; GitHub OAuth optional	Simple for week 1
Testing	Vitest or Jest; Supertest for API; Testcontainers or a local Postgres for integration	Fast unit tests, real-DB integration tests
Quality	ESLint + dependency-cruiser (or `eslint-plugin-boundaries`)	Enforces the layer rules automatically
External	`@octokit` for GitHub; PayPal REST via official SDK or plain HTTP	Hidden behind adapters
Jobs (later)	BullMQ + Redis	Not needed in week 1; outbox poller is enough

3. Architecture: modular monolith with hexagonal layers

One deployable application, divided into modules. Inside each module, dependencies point **inward only**.

```
HTTP routes (Fastify)        thin: validate, call use case, shape response
        ↓
Application (use cases)      orchestrates work; owns the transaction
        ↓
Domain (entities, rules)     pure JS classes, events, state machine
        ↑
Ports (interfaces)           what the app needs from the outside world
        ↑
Infrastructure (adapters)    Knex repositories, PayPal, GitHub, clock
```

The dependency rule: the domain imports nothing outside itself. Application imports domain and ports. Infrastructure imports ports and domain. Only the **composition root** (`main.js`) imports everything and wires concrete classes to ports.

JavaScript has no `interface` keyword, so a port is an abstract base class that throws `NotImplemented`, documented with JSDoc. Test suites for each port ("contract tests") run against both the fake and the real adapter to prove they behave identically.

4. Folder structure

```
src/
  shared/        Money, Clock, EventBus, UnitOfWork base, errors, ids
  identity/      domain/ application/ ports.js infrastructure/ http/
  projects/      milestones, spec versions, criteria, acceptances
  submissions/   developer submissions (commit SHA, staging URL)
  verification/  audit runs, evidence, Verifier strategies
  payments/      payment records, gateway port, idempotency
  audit/         append-only hash-chained log
  main.js        composition root: builds and wires everything
  server.js      starts Fastify
tests/
  unit/ integration/ contract/ e2e/
migrations/
```

Each module uses the same internal layout so any developer can navigate any module.

5. Modules and ownership

Module	Owns	Exposes	Reacts to
identity	users, credentials, sessions	current-user lookup	none
projects	projects, members, milestones, spec versions, criteria, acceptances, decisions	create/accept/fund/review use cases	`SubmissionReceived`, `PaymentCompleted`
submissions	submissions	submit use case	none
verification	audit runs, evidence items, verifiers	run-verification use case	`SubmissionReceived`

Module	Owns	Exposes	Reacts to
payments	payment records, webhook events, gateway adapter	initiate/confirm use cases	`MilestoneFunded` request, `MilestoneApproved`
audit	audit log	append-only write	all domain events
shared	value objects and infrastructure primitives	everything	none

Rule: **a module may call another module's use case or listen to its events, but never import its repositories or touch its tables.**

6. OOP patterns and where they apply

Pattern	Applied to	JavaScript note
State	Milestone lifecycle	Transition table plus guard methods; illegal transitions throw `IllegalTransitionError`
Entity / Aggregate	`Project`, `Milestone`, `SpecVersion`	Classes with private fields (`#field`) and behaviour methods; no public setters
Value Object	`Money`, `CommitSha`, `ContentHash`	Immutable (`Object.freeze`), compared by value; money stored as integer minor units
Repository	One per aggregate	Port returns domain objects; Knex adapter maps rows ↔ objects
Unit of Work	Application layer	Wraps a Knex transaction; exposes repositories scoped to it
Command (use case)	One class per business action	`execute(input)` method; constructor receives ports
Strategy	Verifiers	`ManualReviewVerifier`, `CiResultVerifier` now; agent verifiers later
Factory	`VerifierFactory`, entity factories	Maps a criterion's verification method to a verifier

Pattern	Applied to	JavaScript note
Observer / Domain Events	Cross-module reactions	Entities collect events; dispatcher publishes after commit
Transactional Outbox	Reliable event delivery	Events saved in the same transaction as the state change; a poller dispatches them
Adapter / Gateway	PayPal, GitHub	Translate provider payloads into domain types
Decorator	Idempotency, logging, authorisation around use cases	Wraps a use case with the same `execute` interface

Guidance: do not create an interface for something that will only ever have one implementation. Prefer composition over inheritance; use inheritance only for a small shared base such as `Verifier` or `DomainEvent`.

7. Key design decisions

7.1 Verification as a port (this is what makes agents pluggable)

- A `Verifier` takes a criterion and a submission and returns an `EvidenceResult` (status, explanation, artifact references).
- **Week 1 implementations:** `ManualReviewVerifier` (client marks each criterion met / not met with a note) and `CiResultVerifier` (reads test results uploaded by a GitHub Actions workflow and maps them to criteria by test name or tag).
- **Later:** coverage, mutation, browser, security, anti-gaming and provenance agents are additional `Verifier` classes registered in the factory.
- Each criterion stores its `verificationMethod`; the factory chooses the verifier. No domain code changes when an agent is added.

7.2 Milestone state machine

Draft → Agreed → Funded → In progress → Submitted → In review → Approved → Payment initiated → Paid, with side paths Changes requested (back to Submitted), Disputed and Cancelled. Transitions live in one table inside the domain; every other layer calls entity methods such as `approve(userId)`. A version number on the row gives optimistic locking so two concurrent requests cannot both succeed.

7.3 Spec versioning and agreement

- A spec version is immutable. Editing creates a new version and clears prior acceptances.
- Funding is only allowed when **both** parties have accepted the **current** version (stored with user, timestamp and content hash).
- Verification always records which spec version and commit SHA it ran against.

7.4 Events and the outbox

Use cases save state changes and outgoing events in one database transaction. A small dispatcher reads unsent outbox rows and publishes them to in-process subscribers (payments reacts to approval; audit records everything). This avoids the failure where the state saves but the event is lost.

7.5 Payments

- `PaymentGateway` port with `FakeGateway` (tests) and `PayPalSandboxGateway` (demo).
- Every outbound call carries an idempotency key stored in a unique column.
- Inbound PayPal webhooks are signature-verified and de-duplicated by provider event id; state changes come from the verified webhook, not from a browser redirect.
- A payout can only start from the Approved state. No verifier or future agent has any dependency on the payments module.

7.6 Errors, validation and auth

- Domain throws typed errors (`IllegalTransitionError` , `NotAuthorizedError` , `ValidationError`); one HTTP error handler maps them to status codes.
- Request validation at the edge; business invariants inside entities.
- Authorisation ("is this user the client of this project?") is a use-case concern, expressed once and reused.

7.7 Audit log

Append-only table; each row stores the hash of the previous row. Subscribes to domain events; nothing in the system updates or deletes these rows.

8. Data model (PostgreSQL)

Table	Key points
`users`	unique email, password hash
`projects`, `project_members`	role per project: client or developer
`milestones`	amount (integer minor units), currency, `state`, `version`
`spec_versions`	milestone id, version number, content hash; immutable
`criteria`	belongs to a spec version; description, verification method, severity, blocking flag
`spec_acceptances`	user, spec version, timestamp
`submissions`	milestone, commit SHA, optional staging URL, status
`audit_runs`, `evidence_items`	evidence is insert-only; artifact link, SHA-256 hash, result
`decisions`	approve / changes requested / dispute, comment, linked criteria
`payments`	provider reference, unique idempotency key, status
`webhook_events`	unique provider event id
`outbox_events`	unsent events written with the state change
`audit_log`	append-only with previous-hash column

Conventions: UUID primary keys, foreign keys everywhere, unique constraints for idempotency, `created_at` on every row, and no updates or deletes on evidence or audit tables. Migrations are versioned and run in CI.

9. API surface (week 1)

Area	Endpoints
Auth	register, login, current user

Area	Endpoints
Projects	create, get, invite counterparty, accept invite
Milestones	create, list, get with current spec
Spec	create new version, list versions, accept version
Funding	start funding, PayPal webhook receiver
Work	start work, submit (commit SHA, staging URL), list submissions
Verification	GitHub webhook receiver, upload CI results, run verification, list evidence
Review	manual criterion review, approve, request changes, dispute
Audit	timeline for a milestone, evidence bundle export (JSON)

All routes are thin: validate input, call one use case, return a response object.

10. Testing strategy

- **Unit tests** for entities, state machine, value objects and use cases using fakes (target most coverage here; they run in milliseconds).
- **Contract tests** per port, run against both the fake and the real adapter.
- **Integration tests** against a real Postgres for repositories, unit of work and the outbox.
- **API tests** with Supertest for each endpoint including permission failures.
- **One end-to-end test** that runs the full happy path and one rejection path.
- Architecture tests with dependency-cruiser fail the build if a layer or module boundary is violated.

11. Security basics for week 1

- Hash passwords with argon2 or bcrypt; secure session or short-lived JWT.
- Verify GitHub and PayPal webhook signatures; reject replays.
- Secrets only from environment variables; never committed.
- Parameterised queries only (Knex does this); rate-limit auth and webhook routes.
- Treat all repository content as untrusted data. Week 1 does not execute submitted code at all; any future execution happens in isolated sandboxes.

12. Week 1 plan (13–19 Oct)

Day	Focus	Done when
Tue 13	Repo, open-source licence, lint, test runner, CI, folder skeleton, shared kernel (Money, errors, ids, Clock), DB connection, first migration	Empty app boots; tests and lint run in CI
Wed 14	Identity module; milestone entity and state machine; repository ports; Knex repositories and Unit of Work	State machine fully unit-tested; illegal transitions rejected
Thu 15	Projects use cases: create, invite, spec versions, criteria, dual acceptance, funding gate	Cannot fund without both acceptances; any edit creates a new version
Fri 16	Submissions; verification port with manual and CI-result verifiers; GitHub webhook with signature check; evidence storage	A commit with uploaded CI results produces evidence rows
Sat 17	Review use cases (approve, request changes, dispute); outbox and dispatcher; audit log with hash chain	Approval emits an event consumed by other modules
Sun 18	Payments: gateway port, fake adapter, PayPal Sandbox adapter, idempotency, webhook de-duplication, reconciliation	Sandbox payment moves milestone to Paid; replayed webhook changes nothing
Mon 19	End-to-end test, architecture checks, error handling, API docs, seed data, README; buffer for slipped work	Full happy path and rejection path pass from a clean checkout

Cut order if behind: dispute path → evidence export → reconciliation job → GitHub OAuth. Do not cut the state machine, spec versioning, idempotency or the audit log.

13. Definition of done for week 1

1. A client and a developer can complete one milestone from agreement to a sandbox payment using only the API.
2. All illegal transitions and unauthorised actions are rejected with tested errors.
3. Replaying any webhook or payment request causes no duplicate effect.
4. Verification works through the `Verifier` port with manual and CI-result strategies.

5. Architecture checks pass, proving the domain imports nothing from infrastructure.

14. Plug-in points for later weeks

Future feature	Where it plugs in	Core changes needed
Coverage, mutation, browser, security, anti-gaming, provenance agents	New `Verifier` classes registered in the factory	None
Spec compiler (AI-drafted criteria)	New `SpecDraftingService` port used by the create-spec use case	Add one optional use-case step
Evidence reporter	Subscriber to `VerificationCompleted`	None
Job queue for long audits	Replace outbox poller with BullMQ worker	Adapter only
Production payments	New gateway adapter after legal review	Adapter only

15. Open decisions

1. Fastify or Express (recommendation: Fastify).
2. Knex versus another data-access library (recommendation: Knex for explicit mapping).
3. Plain JavaScript with JSDoc versus moving to TypeScript after the hackathon.
4. Hosting for the demo (managed Postgres plus a single Node service is enough).